Reproducing Reasoning Emergence

in Small LLMs via GRPO Post-Training

A reproduction of DeepSeek-R1's RL pipeline at sub-3B scale using open-source tools

Model: Qwen2.5-1.5B · Dataset: NuminaMath-CoT + GSM8K · Hardware: Google Colab T4 GPU

Evaluation Benchmark: GSM8K Test Set

By: Sai Akshitha Boddupalli and Keyaba Gohil

Motivation: DeepSeek-R1 at Small Scale

The DeepSeek-R1 Finding

- **DeepSeek-R1 showed that RL from verifiable rewards alone — without human-labeled chain-of-thought — can elicit structured reasoning in large LLMs**
- Used GRPO (Group Relative Policy Optimization) to teach models to think step-by-step
- Reward signal: correct final answer + format adherence (<think> tags)
- Result: models spontaneously develop self-checking, backtracking, and multi-step reasoning

Our Research Question

- **Can the same post-training paradigm work at sub-3B scale?**
- Using only open-source tools and public datasets?
- Model: Qwen2.5-1.5B — small enough for a single T4 GPU (15 GB VRAM)
- Benchmark: GSM8K — the standard elementary math reasoning test
- Goal: reproduce the SFT → RL accuracy jump and understand the tradeoffs

DESIGN

Model & Dataset Selection-Why These Choices?

Qwen2.5-1.5B Base Model

- 1.5B params fits in T4 VRAM in fp16 (≈3 GB) — critical for Colab
- Open-source (Apache 2.0), HuggingFace-native
- Pretrained on math and code, so we are not starting from scratch

GSM8K — GRPO Training & Eval

- 7,473 grade-school math problems with verifiable numeric answers
- Used 500 training samples for GRPO — large enough for signal, small enough for T4
- Verifiable reward makes it ideal for RL — no LLM judge needed

NuminaMath-CoT — SFT Dataset

- Competition-grade math problems with full CoT solutions
- Used 2,000 samples; formatted with <think> tags to prime reasoning format
- Diverse difficulty hence generalises better than GSM8K

Hardware: Google Colab T4 GPU

- NVIDIA T4 — 15 GB VRAM, free tier accessible
- fp16 training + LoRA adapters kept memory under budget
- Additional runs on RunPod (A-series GPU) for KL=0.1 ablation

STAGE 1

SFT Warm-up: Stabilising Generation Before RL

Why SFT Before RL?

- Base LLMs often produce incoherent or repetitive outputs on math prompts
- SFT on high-quality CoT examples teaches the model the expected output format (<think> tags, step-by-step structure)
- GRPO requires a well-initialized, stable policy with competent CoT behavior, otherwise sampled trajectories are low-quality, rewards are sparse or degenerate, and policy gradient estimates have high variance or vanish.

SFT Result

8%

Baseline (0-shot)

↓

28%

After SFT (5 epochs)
GSM8K test set

+20 pp gain

SFT Configuration

Parameter	Value	Reason
Dataset	NuminaMath-CoT, 2,000 samples	Competition math → harder than GSM8K → better generalisation
Epochs	5	Convergence on T4 without overfitting
Learning rate	2e-4	Standard for LoRA fine-tuning
LoRA rank r	16, alpha=32	~3% trainable params; fits T4 memory
Batch / GradAcc	2 / 8 (eff. 16)	Memory vs. gradient quality tradeoff
Max seq length	512 tokens	Balances CoT length & VRAM
Format	<think>…</think> + Final Answer	Primes model for GRPO format reward

STAGE 2

GRPO Implementation: Algorithm & Reward Design

How GRPO Works

- For each prompt, sample G=2 completions from the current policy
- Score each completion with the reward function r(y)
- Advantage = (r - mean(r)) / std(r) — relative within the group
- Policy gradient update: maximise expected advantage, penalised by KL divergence from SFT policy

Reward Function (Combined)

- Correctness reward: +1.0 if predicted number = ground truth (exact match)
- Format reward: +0.3 for step-by-step reasoning (length > 20 words, keywords, contains numbers)
- Max reward per completion: 1.3

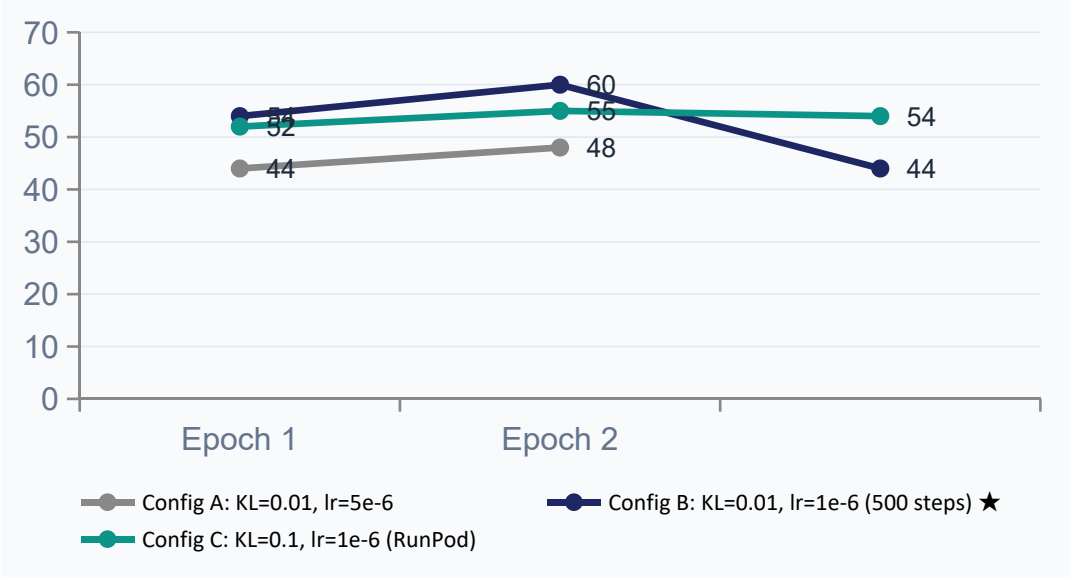
STAGE 2

GRPO Implementation: Parameters

Parameter	Value	Reason
Base model	Qwen2.5-1.5B + SFT LoRA	Starts from an SFT-aligned policy with learned CoT structure; LoRA keeps RL updates parameter-efficient and stabilizes training by constraining drift.
GRPO samples	500 (GSM8K train)	Small, high-quality dataset is sufficient due to verifiable rewards; prioritizes signal quality over scale for policy optimization.
Epochs	3	Limits over-optimization on a small dataset; RL can quickly overfit or collapse, so fewer passes maintain generalization.
Batch / GradAcc	4 / 8 (eff. 32)	Achieves a larger effective batch size for lower-variance gradient estimates while staying within memory constraints.
Max prompt length	256 tokens	Covers most GSM8K problem statements while avoiding unnecessary context that increases compute and noise.
Max generation	128,256 tokens	128 tokens covers simple single-step problems to test baseline RL stability, while 256 tokens provides enough space for the 4–6 arithmetic steps that GSM8K problems typically require to reach a verifiable final answer.
Num generations (G)	2,4	Enables relative comparison (ranking/advantage estimation) across sampled outputs while keeping compute cost low.
LR scheduler	Cosine + 5% warmup	Warmup prevents early instability; cosine decay ensures smooth convergence and avoids late-stage overshooting.
Precision	fp16	Reduces memory footprint and increases throughput with minimal impact on training stability for this scale.

Main Results: GSM8K Accuracy Across Training Stages

8%	28%	60%	+52pp
Baseline (zero-shot)	After SFT (5 epochs)	Best GRPO (Config B ep2)	Total accuracy gain



Reading the Chart

- **All GRPO configs surpass SFT ceiling of 28% by epoch 1**
- Config B peaks at 60% — 2.1× the SFT result
- Config B collapses at epoch 3 (44%) — reward hacking signal
- Config C plateaus at 52-55% — stable but lower ceiling
- Lower LR + tuned KL is the key to peak performance

RL Design Choices: Ablation Study

Config	No. of Generation(G)	Max Generation Length	KL β	Learning Rate	Steps	Epoch 1 Acc.	Epoch 2 Acc.	Epoch 3 Acc.
A (Colab)	2	128	0.01	5e-6	500	44%	**48%**	--
B (Colab) ★	2	256	0.01	1e-6	500	54%	**60%** ★	44%
C (RunPod)	4	256	0.1	1e-6	500	52%	55%	54%

Learning Rate Finding

- 5e-6 (Config A) produced unstable updates — policy changed too fast per step, early saturation at 48%
- 1e-6 (Configs B & C) gave finer-grained updates, letting the policy find better optima

KL Penalty Finding

- Low KL (0.01, Config B): allows more exploration → higher peak, but risks policy collapse at epoch 3
- High KL (0.1, Config C): constrains policy near SFT baseline → lower peak but stable across all 3 epochs

ANALYSIS

Reward Hacking & Training Stability

Key Observation: Config B (best peak) collapses from 60% → 44% at epoch 3

This is a textbook example of reward hacking — the policy exploits the format reward without improving answer quality.

What Causes Policy Collapse?

- Format reward (+0.3) is easier to exploit than correctness reward (+1.0)
- Model learns to produce long outputs with step-keywords regardless of whether reasoning is correct
- With KL=0.01, the policy drifts far from the SFT baseline — losing the grounded reasoning from warm-up
- By epoch 3, token distribution shifts enough to hurt answer extraction — numbers appear in wrong positions

How Higher KL Prevents This

- KL=0.1 (Config C) adds a stronger regularisation penalty for deviating from the SFT policy
- The policy cannot exploit format reward as easily — it must stay near a policy that already answers correctly
- Stability across all 3 epochs (52%→55%→54%) at the cost of ~5pp peak accuracy
- Tradeoff: exploration vs. stability — classic RL dilemma, confirmed empirically here

EXAMPLES

Reasoning Quality: Before vs. After Post-Training

Problem: Kylar went to the store to buy glasses for his new apartment. One glass costs $5, but every second glass costs only 60% of the price. Kylar wants to buy 16 glasses. How much does he need to pay for them?

Baseline (8%)

```
================================================================
BASELINE (no fine-tuning)
================================================================
 Step 1: Determine the price of the first glass.
The first glass costs $5.

Step 2: Determine the price of the second glass.
The second glass costs 60% of the price of the first glass.
To find 60% of $5, we multiply $5 by 0.60:
$5 * 0.60 = $3

Step 3: Determine the price of the third glass.
The third glass also costs 60% of the price of the first glass.
To find 60% of $5, we multiply $5 by 0.60:
$5 * 0.60 = $3

Step 4: Determine the price of the fourth glass.
The fourth glass also costs 60% of the price of the first glass.
To find 60% of $5, we multiply $5 by 0.60:
$5 * 0.60 = $3

Step 5: Determine the price of the fifth glass.
The fifth glass also costs 60% of the price of the first glass.
To find 60% of $5, we multiply $5 by 0.60:
$5 * 0.60 = $3

Step 6: Determine the price of the sixth glass.
The sixth glass also costs 60% of the price of the first glass.
To find 60% of $5, we multiply $5 by 0.60:
$5 * 0.60 = $3
```

No arithmetic shown, Wrong answer, Vague, incoherent

After SFT (28%)

```
================================================================
SFT MODEL (NuminaMath-CoT fine-tuned)
================================================================
 <think>
To solve this problem, we need to calculate the cost of buying one
glass that is 60% of the original price and then extend this to 16 glasses.

1. **Calculate the cost of one discounted glass:**
   - Original price of one glass = \$5
   - Discounted price = 60% of \$5
   \[
   \text{Discounted price} = 0.60 \times 5 = \$3
   \]

2. **Calculate the total cost for 16 glasses:**
   - Price per discounted glass = \$3
   - Total number of glasses = 16
   \[
   \text{Total cost} = 16 \times 3 = \$48
   \]

3. **Conclusion:**
   - Therefore, Kylar needs to pay a total of \$48 for 16 glasses.

\[
\boxed{48}
\]
</think>
Final Answer: 48
```

Uses <think> tags, Shows arithmetic, Wrong final answer

After GRPO (60%)

```
================================================================
GRPO MODEL (RL fine-tuned)
================================================================
 <think>
To solve this problem, we need to calculate the cost of each
glass and then sum up the total cost for 16 glasses.

1. First, let's find the cost of every second glass. Since every second
glass costs 60% of the price, the cost of every second
glass is $5 * 0.6 = $3.

2. Next, let's find the cost of the remaining 8 glasses. Since all the
glasses before the second glass are at the full price of $5, the cost
of these 8 glasses is $5 * 8 = $40.

3. Now, let's calculate the total cost for 16 glasses. We have 8 glasses
at the full price and 8 glasses at 60% price, so the total cost is
$40 + $3 * 8 = $40 + $24 = $64.

4. Finally, let's sum up the costs for all 16 glasses. The total cost is
$64.

Therefore, Kylar needs to pay $\boxed{64}$ dollars for 16 glasses.
</think>
Final Answer: 64
```

Multi-step chain of thought, Applies all problem constraints, Correct final answer: $64